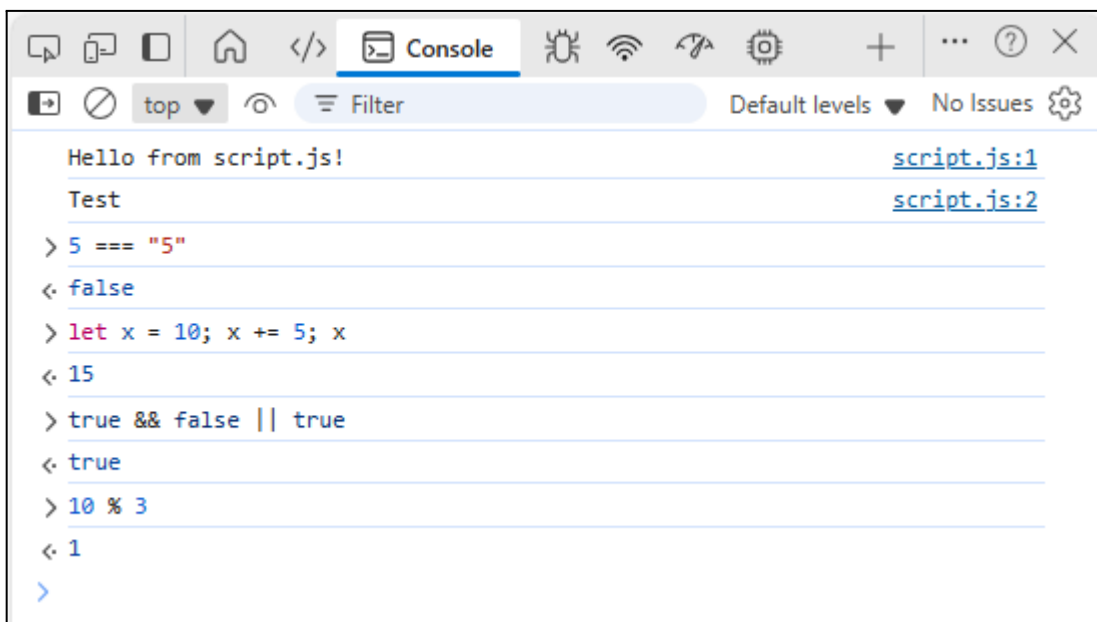


JavaScript Practical Assignment

Part 0 — Quick Recap Warm-Up

Predicted vs. actual output

Code	Predicted	Actual
<code>typeof (10 + "5")</code>	String	String
<code>5 === "5"</code>	False	False
<code>let x = 10; x += 5; x</code>	15	15
<code>true && false true</code>	True	True
<code>10 % 3</code>	1	1



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following output:

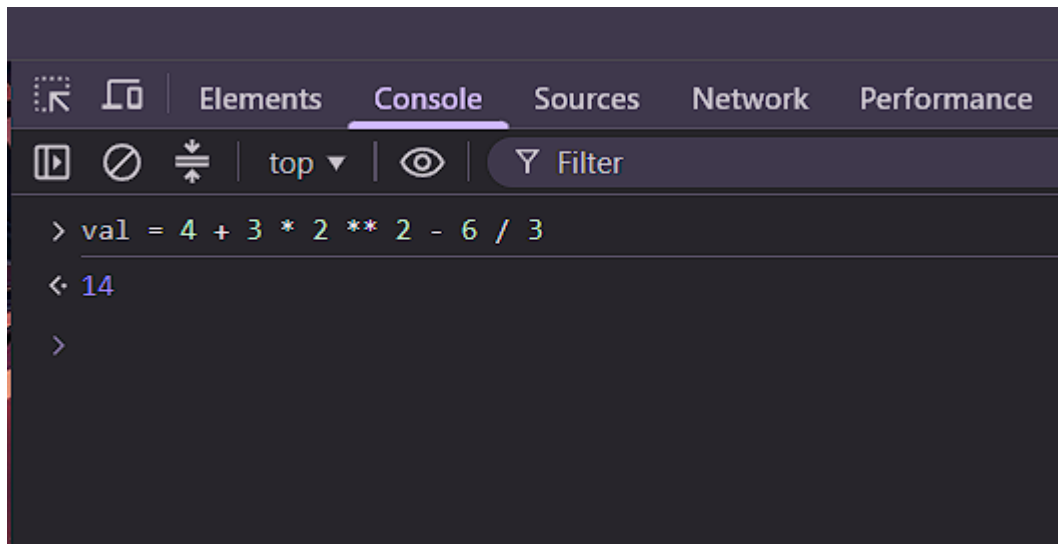
```
Hello from script.js!
Test
> 5 === "5"
< false
> let x = 10; x += 5; x
< 15
> true && false || true
< true
> 10 % 3
< 1
>
```

Part 1 — Operator Precedence: Level Up

Task 1.1 — Trace It Yourself: let `val = 4 + 3 * 2 ** 2 - 6 / 3`; Predict the value by writing out each step (like the guided example above), THEN verify in the console.

Steps

```
// Step 1: 2 ** 2 = 4
// Step 2: 3 * 4 = 12
// Step 3: 6 / 3 = 2
// Step 4: 4 + 12 = 16
//Step 5: 16 - 2 = 14
```

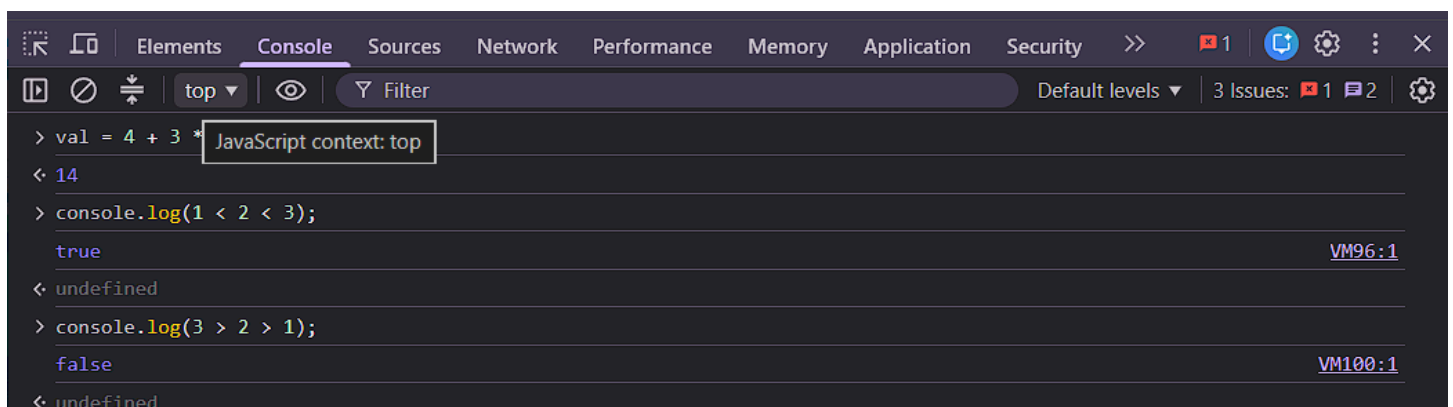


A screenshot of a web browser's developer console. The 'Console' tab is selected. The input is `> val = 4 + 3 * 2 ** 2 - 6 / 3` and the output is `< 14`.

Task 1.2 — Comparison Chaining Gotcha: Predict, then run, both lines below. The second result surprises most people — explain why in 2–3 sentences, tracing left to right.

Explanation:

- 1 st line will result true because js follows left to right method for comparison
- 2 nd line will result false because when $3 > 2 = \text{true}$ $\text{true} = 1$, $1 > 1$ is false.



A screenshot of a web browser's developer console. The 'Console' tab is selected. The input is `> val = 4 + 3 *` and the output is `< 14`. Below this, two lines of code are entered: `> console.log(1 < 2 < 3);` and `> console.log(3 > 2 > 1);`. The first line outputs `true` and the second line outputs `false`. A tooltip 'JavaScript context: top' is visible over the first line of code.

Task 1.3: Write ONE expression that combines at least 4 different operators (mixing arithmetic and comparison) and evaluates to true. Show your reasoning, step by step, like the guided example.

Step 1: $3 * 2 = 6$
Step 2: $5 + 6 = 11$
Step 3: $11 > 10 = \text{true}$

```
> console.log(5 + 3 * 2 > 10);  
true  
← undefined
```

[VM141:1](#)

Part 2 — Nested Ternary Decision Logic

Task 2.1 — Movie Ticket Pricing: Build a nested ternary: age < 5 → "Free", age < 12 → "₹100", age < 60 → "₹250", else (senior citizen) → "₹150". Test it with 5 different ages.

Age - 25

```
> let age = 25;  
  
let ticketPrice = age < 5 ? "Free"  
                  : age < 12 ? "₹100"  
                  : age < 60 ? "₹250"  
                  : "₹150";  
  
console.log(ticketPrice);
```

₹250

[VM240:8](#)

Age - 3

```
> let age = 3;  
  
let ticketPrice = age < 5 ? "Free"  
                  : age < 12 ? "₹100"  
                  : age < 60 ? "₹250"  
                  : "₹150";  
  
console.log(ticketPrice);
```

Free

[VM248:8](#)

Age - 8

```
> let age = 8;

let ticketPrice = age < 5 ? "Free"
                  : age < 12 ? "₹100"
                  : age < 60 ? "₹250"
                  : "₹150";

console.log(ticketPrice);

₹100 VM256:8
← undefined
```

Age - 59

```
> let age = 59;

let ticketPrice = age < 5 ? "Free"
                  : age < 12 ? "₹100"
                  : age < 60 ? "₹250"
                  : "₹150";

console.log(ticketPrice);

₹250 VM263:8
```

Age - 65

```
> let age = 65;

let ticketPrice = age < 5 ? "Free"
                  : age < 12 ? "₹100"
                  : age < 60 ? "₹250"
                  : "₹150";

console.log(ticketPrice);

₹150 VM270:8
```

Task 2.2 — Shipping Cost Calculator: `cartTotal >= 2000` → "Free Shipping", `cartTotal >= 1000` → "₹50", else → "₹100". Test with 3 different cart totals.

Cart Total 1500

```
> let cartTotal = 1500;

let shippingCost = cartTotal >= 2000 ? "Free Shipping"
                  : cartTotal >= 1000 ? "₹50"
                  : "₹100";

console.log(shippingCost);

₹50
```

VM274:7

Cart Total 2500

```
> let cartTotal = 2500;

let shippingCost = cartTotal >= 2000 ? "Free Shipping"
                  : cartTotal >= 1000 ? "₹50"
                  : "₹100";

console.log(shippingCost);

Free Shipping
```

VM281:7

Cart Total 500

```
> let cartTotal = 500;

let shippingCost = cartTotal >= 2000 ? "Free Shipping"
                  : cartTotal >= 1000 ? "₹50"
                  : "₹100";

console.log(shippingCost);

₹100
```

VM287:7

Task 2.3 — Trickier: Ternary + Logical Operator Together: Write a SINGLE ternary expression where discountType is "VIP Discount" if (isMember AND totalSpent >= 5000), "Regular Discount" if (totalSpent >= 5000 but not a member), or "No Discount" otherwise. Use && inside the ternary condition.

```
let isMember = true;
let totalSpent = 6000;

let discountType = (isMember && totalSpent >= 5000)
  ? "VIP Discount"
  : totalSpent >= 5000
  ? "Regular Discount"
  : "No Discount";

console.log(discountType);
```

VIP Discount

undefined

[VM291:10](#)

Part 3 — Manual Aggregation: Arrays & Objects without Loops

Task 3.1: Given an array of 5 exam scores of your choice, manually compute the sum and average using indexing. Then use a ternary to classify the average as "Pass" (>= 40) or "Fail".

```
> const scores = [65, 72, 48, 80, 55];

const sum = scores[0] + scores[1] + scores[2] + scores[3] + scores[4];

const average = sum / scores.length;

const result = average >= 40 ? "Pass" : "Fail";

console.log("Sum:", sum);
console.log("Average:", average);
console.log("Result:", result);
```

Sum: 320

Average: 64

Result: Pass

[VM333:9](#)

[VM333:10](#)

[VM333:11](#)

undefined

Task 3.2: Add a 4th product object of your choice, calculate its subtotal, and recompute the grand total by including it.

```
> const item1 = { name: "Notebook", price: 60, qty: 3 };
const item2 = { name: "Pen", price: 10, qty: 5 };
const item3 = { name: "Bag", price: 800, qty: 1 };
const item4 = { name: "Pencil", price: 20, qty: 4 };

const subtotal1 = item1.price * item1.qty;
const subtotal2 = item2.price * item2.qty;
const subtotal3 = item3.price * item3.qty;
const subtotal4 = item4.price * item4.qty;

const grandTotal = subtotal1 + subtotal2 + subtotal3 + subtotal4;

console.log("Subtotal 4:", subtotal4);
console.log("Grand Total:", grandTotal);

Subtotal 4: 80
Grand Total: 1110
< undefined
```

Part 4 — Deep Dive: Tricky Type Coercion

Task 4.1: For all 7 lines above, record the actual output and a one-line reason in your lab record. Circle the one you find most surprising and explain why in 2–3 sentences.

Expression	Actual Output	Reason
"5" + 3	"53"	+ joins strings, so 3 is converted to a string.
"5" - 3	2	- converts the string "5" into a number.
"abc" * 2	NaN	"abc" cannot be converted into a valid number.
NaN === NaN	False	NaN is not equal to itself in JavaScript.
[] == false	True	Both sides are converted to compatible primitive values (0).

"10" == 10	True	== performs type coercion and converts "10" to 10.
null + 1	1	In numeric addition, null is converted to 0.

Most Surprising: NaN === NaN

```
> console.log(NaN === NaN);
false
VM341:1
< undefined
```

NaN means "Not-a-Number," but it is actually a special numeric value in JavaScript. By definition, NaN is not equal to any value, including another NaN, so NaN === NaN returns false

Task 4.2: Predict, then run, typeof NaN. Explain why this result seems contradictory given that "NaN" stands for "Not-a-Number."

```
> typeof NaN
< 'number'
>
```

NaN means "Not a Number", but it is actually a special value inside JavaScript's number data type

Part 6 — Debugging Challenge: Multi-Bug Snippets

Snippet 1

```
> let price = 500;
   let discount = price * 0.1;
   discount = discount + 5;

   let final = price - discount;

   console.log("Final: " + fnal);

✖ ▶ Uncaught ReferenceError: fnal is not defined
   at <anonymous>:7:25
VM355:7
```

Discount is declared using const so it cannot be reassigned that why it is showing error.

Snippet 2 (silent bug — no crash on the first bug!)

```
> let marks = "85";  
let result = marks === 85 ? "Pass" : "Fail";  
console.log(Result);
```

✖ Uncaught SyntaxError: Identifier 'result' has already been declared (at VM359:1:1)

VM359:1

While printing the result in `console.log` the `Result` is used which is not equal to `result===Result`.

Snippet 3

```
> const cartTotal = 1200;  
  
cartTotal = cartTotal - 100;  
  
let shipping = cartTotal >= 1500 ? "Free" : "Paid";  
  
console.log(shipping);
```

✖ Uncaught SyntaxError: Identifier 'cartTotal' has already been declared (at VM378:1:1)

VM378:1

`Const` can not be reassigned and `:` is not present in ternary operator.